

DRPY: Pinpointing Inefficient Memory Usage in Multi-Layer Python Applications

Jinku Cui*, Qidong Zhao[†], Yueming Hao[‡], and Xu Liu[§]
North Carolina State University

Raleigh, USA

*jcui23@ncsu.edu, [†]qzhao24@ncsu.edu, [‡]yhao24@ncsu.edu, [§]xliu88@ncsu.edu

Abstract—Python has become an increasingly popular programming language, especially in the areas of data analytics and machine learning. Many modern Python packages employ a multi-layer design: the Python layer manages various packages and expresses high-level algorithms; the native layer is written in C/C++/Fortran/CUDA for efficient computation. Typically, each layer manages its own computation and memory and exposes APIs for cross-layer interactions. Without holistic optimization, performance inefficiencies can exist at the boundary between layers.

In this paper, we develop DRPY, a novel profiler that pinpoints such memory inefficiencies across layers in Python applications. Unlike existing tools, DRPY takes a hybrid and fine-grained approach to track memory objects and their usage in both Python and native layers. DRPY correlates the behavior of memory objects across layers and builds an object flow graph to pinpoint memory inefficiencies. In addition, DRPY captures rich information associated with object flow graphs, such as call paths and source code attribution to guide intuitive code optimization. Guided by DRPY, we are able to optimize many Python applications with non-trivial performance improvement. Many optimization patches have been validated by application developers and committed to application repositories.

I. INTRODUCTION

Today, Python is becoming the mainstream programming language for the domains of data analysis and machine learning. State-of-the-art Python applications, such as PyTorch [1], TensorFlow [2], and Jax [3], typically employ a multi-layer design, as shown in Figure 1. These layers can typically be divided into two levels: the high-level layer (called the *Python layer*), including Python application code and packages, focuses on algorithm design, which provides an abstraction to hide most implementation details for better code readability and maintenance; the low-level layer (called the *native layer*), including various high-performance libraries, performs efficient computation supported by C/C++/Fortran codes or accelerated by CUDA kernels. Such programming models have proven effective in improving the productivity of both domain and library developers. For example, machine learning engineers can develop powerful models in Python with minimal efforts; library developers can focus on developing necessary numerical operations that are fully optimized for hardware features.

However, such a multi-layer programming model logically divides up the entire software stack. As shown in Figure 1, the layers are disjoint from each other in design and implementation, and they typically evolve individually. Without holistic optimization, inefficient interactions can exist at the boundary

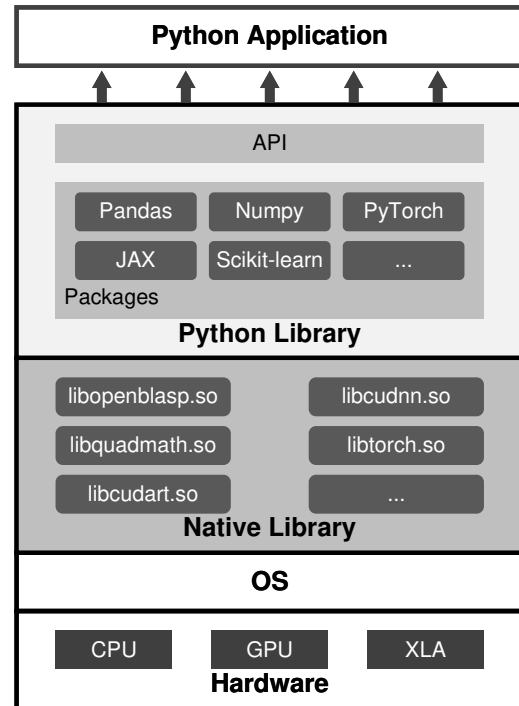


Fig. 1. Software layers involved in typical Python applications.

between layers. One such inefficiency we observe is in the memory subsystem. Since each layer typically manages its own memory, sharing data across layers can easily lead to unnecessary memory operations, which we refer to as cross-layer memory inefficiencies.

A. Motivating Examples

Listing 1 shows an example of cross-layer memory inefficiencies between PyTorch (Python layer) and NumPy (Native layer). This code snippet comes from the Super SloMo model [4]. It uses `np.meshgrid` to generate coordinate matrices stored in two NumPy arrays. However, its implementation is inefficient. It first creates two NumPy arrays `gridX` and `gridY` in line 4, and then creates the two PyTorch tensors based on these NumPy arrays in lines 7 and 8. The use of the NumPy arrays `gridX` and `gridY` in this function adds a significant amount of memory overhead.

```

1 def __init__(self, W, H, device):
2     ...
3     # create a grid
4     gridX, gridY = np.meshgrid(np.arange(W),
5                                np.arange(H))
6     self.W = W
7     self.H = H
8     self.gridX = torch.tensor(gridX,
9                                requires_grad=False,
10                               device=device)
11    self.gridY = torch.tensor(gridY,
12                               requires_grad=False,
13                               device=device)
14+    self.gridX, self.gridY = torch.meshgrid(
15+        torch.arange(W, requires_grad=False,
16+                      device=device),
17+        torch.arange(H, requires_grad=False,
18+                      device=device),
19+        indexing='xy')

```

Listing 1. Memory overuse in the interaction between PyTorch and Numpy. Optimization yields a 25× speedup and 50% memory usage reduction.

With our investigation, these two NumPy arrays are not necessary. As an optimization, we avoid creating NumPy arrays and directly create PyTorch tensors using the PyTorch API `torch.meshgrid`. This optimization reduces memory consumption by 50% and gives a 25× speedup to this function. The optimization patch has been upstreamed to the PyTorch repository.

We name the performance inefficiency in Listing 1 as *cross-layer memory inefficiency* because it involves multiple layers—PyTorch and NumPy.

It is worth noting that cross-layer memory inefficiencies exist not only at the boundary between Python and native layers, but also within Python or native components. Section III shows a detailed categorization.

a) Challenges of Cross-Layer Memory Inefficiency Analysis: Identifying and optimizing cross-layer memory inefficiencies requires analysis across multiple layers, a task known for its inherent difficulty. On the one hand, the Python layer expresses algorithms, concealing the majority of the low-level execution details through the runtime system (e.g., interpreter and garbage collector). On the other hand, the native layer forfeits much of the semantic information of the program, leading to the challenge of correlating execution behavior with high-level algorithms. A novel analysis is necessary to pinpoint cross-layer memory inefficiencies in Python applications.

B. Existing Solutions

Profilers play an important role in identifying performance inefficiencies. However, traditional profilers fail to detect memory overuse across layers: (i) *Traditional Python profilers*, such as cProfile [5], guppy3 [6], PySpy [7], PyFlame [8], PyInstrument [9], PyCallGraph [10], PProfile [11], MemoryProfiler [12] and Austin [13], monitor execution using Python interpreter APIs. They create profiles by capturing only Python runtime information. These profilers assign various metrics (e.g., execution time and memory usage) to Python constructs, but provide limited insights into the behavior of native library internals that interact with the operating system and hardware.

(ii) *Traditional native profilers* that work on native binary executables, such as VTune [14], Perf [15], Toddler [16], HPCToolkit [17], Witch [18], RedSpy [19], DeadSpy [20], LoadSpy [21], provide low-level binary execution details. These tools use hardware performance monitoring units (PMUs) or binary instrumentation frameworks to analyze inefficient code generation, but they lack the semantics from high-level Python code.

State-of-the-art Python profilers, such as Scalene [22], PieProf [23], and SnakeViz [24] are able to monitor execution via OS timers or performance counters and associate measurement with both native and Python code. However, these tools do not offer in-depth insights to facilitate optimization for cross-layer memory inefficiencies because they do not track fine-grained memory object accesses that traverse across layers.

C. Our Approach

We propose DRPY, a novel profiler that pinpoints memory inefficiencies across layers in Python applications. The key novelty of DRPY is its unified profiling framework that monitors both Python and native code. DRPY performs fine-grained, cross-layer analysis that is able to correlate memory behavior in different layers and identify inefficiencies with deep insights. In summary, this paper makes the following contributions.

- We conduct a study of memory inefficiencies due to problematic cross-layer interactions in modern Python applications. We also categorize cross-layer memory inefficiencies according to their root causes.
- We design and implement DRPY, the novel profiler to identify cross-layer memory inefficiencies by analyzing fine-grained memory operations across different layers. DRPY provides deep insights such as object flow graphs, call paths, and source code attribution for intuitive optimization guidance.
- We use DRPY to study a number of popular Python projects. Guided by DRPY, we are able to optimize many cross-layer memory inefficiencies for non-trivial speedups. Many of our optimizations have been validated or accepted by the application developers.

The rest of the paper is organized as follows. Section II reviews existing approaches and distinguishes DRPY. Section III delves into the cross-layer memory inefficiencies and categorizes these inefficiencies according to their causes. Sections IV and V describe DRPY methodology and implementation, respectively. Section VI evaluates DRPY by presenting experimental results of DRPY on popular Python applications. Section VII studies several representative cases to show the performance optimization guided by DRPY. Finally, Section VIII presents some conclusions and previews some future work.

II. RELATED WORK

This section compares existing tools that analyze inefficiencies in Python and native code to distinguish DRPY.

A. Python Performance Analysis Tools

Static tools, such as PyExZ3 [25], PySym [26], flake8 [27], and Frosted [28], analyze Python source code and use multiple heuristics to identify code problems [29]. Compiler infrastructures, such as XLA [30], TVM [31], and Harp [32] can optimize deep learning applications. However, all of these approaches overlook the dynamic behavior of Python and miss potential optimization opportunities.

Dynamic profilers provide a complementary approach. cProfile [5] measures Python code execution, offering information about the frequency and time of execution in specific code regions. Guppy [6] uses object-centric profiling, associating metrics like allocation frequency, allocation size, and cumulative memory usage with individual Python objects. PyInstrument [9] and Austin [13] periodically capture Python call stack frames to identify execution/memory hotspots in Python code. PySpy [7] can attach to a Python process and locate function hotspots in real time. Dedicated profilers such as PyTorch Profiler [33], Jax Profiler [34], and TensorFlow Profiler [35] specialize in analyzing Python-based deep learning frameworks. Unlike DRPY, these profilers primarily focus on Python code and do not provide insights into the cross-layer memory inefficiencies between Python and native libraries.

Scalene [22] and PieProf [23], perhaps the most related tools to DRPY, are able to monitor Python/native execution and correlate their behaviors. While they can measure memory usage during program execution, these tools do not monitor the flow of memory objects across Python and native layers to identify problematic objects.

B. Native Performance Analysis Tools

There are a large number of native profilers [14], [15], [17] for analyzing program execution. We review only the most related work that identifies memory inefficiencies. Toddler [16] identifies redundant memory loads across loop iterations and LDoctor [36] reduces Toddler’s overhead by applying dynamic sampling and static analysis. DeadSpy [20], RedSpy [19], and LoadSpy [21] analyze dynamic instructions throughout program execution to detect useless computation or data movement. CIDetector [37], Witch [18], and VClinic [38] unify the detection of redundant memory accesses in individual frameworks for various purposes, such as studying different compiler code generation, reducing measurement overhead, and broadening the applicability to both x86 and ARM workloads. Unfortunately, all of them only monitor native binary and do not identify cross-layer memory inefficiencies as DRPY does.

III. CROSS-LAYER MEMORY INEFFICIENCIES

Python uses a managed memory runtime, which employs a “Copy on Write” (COW) strategy. COW allows the Python runtime to create a copy of an object when this object is to be modified. This strategy can optimize memory usage by avoiding the creation of duplicate objects, especially when working with large Python objects, such as lists or dictionaries. However, COW does not hold when multiple layers are involved, because objects are copied based on the assumption that they will be

modified in other layers. We define the inefficiencies caused by violating the COW rule as cross-layer memory inefficiencies, which typically manifest themselves as redundant memory operations. Cross-layer memory inefficiencies are mainly due to disjoint memory management strategies in different layers. We first categorize cross-layer memory inefficiencies according to their occurrence among different layers and then identify their common symptoms to motivate the design of DRPY.

A. Inefficiencies across Python Packages

This type of inefficiency arises in the interaction between application code and different Python packages. Listing 2, illustrates an example of memory inefficiency between Python packages.

```
def run():
2  arr = np.random.random((10_000_000, 20))
3  df = pd.DataFrame(arr)
4  cols_to_agg = list(df.columns)
5+ pd.set_option("mode.copy_on_write", True)
6  manual_lookup_mean = df.loc[cols_to_agg].mean()
7  ...
```

Listing 2. There exists an unnecessary copy when calculating the average value of each column in the pandas data frame. The `df.loc[]` only needs to read the relevant columns in the data frame. There is no need to aggressively create copies of these columns when calculating the average.

In this Python method, Pandas [39] is used to maintain all data in a data frame `df`. In lines 4 and 6, the code accesses the `df` columns to calculate the average of each column. Pandas employs an implicit in-memory cache to maintain items in the data frame for fast processing. Consequently, Pandas caches each column when accessed in the data frame operation `df.loc[]` on line 6, needlessly doubling the memory usage. However, in this function, there is no need for such in-memory caching since the data in the data frame remains unaltered. To address this, we introduce line 5 to prevent the caching of the data frame’s immutable data. This optimization reduces memory usage by 48% and results in a 9.8× speedup for this function.

B. Inefficiencies between Python and Native Layers

This type of inefficiency occurs at the boundary between the Python and native layers. The native libraries typically do not follow COW because they prefer to convert Python objects to native objects for fast processing.

```
def layers_weights(model, initial=True):
2  network_weights = []
3  ...
4- return numpy.array(network_weights)
5+ return numpy.asarray(network_weights)
```

Listing 3. Code snippet demonstrating unnecessary copying from Python built-in data structures to package-defined class instances. The last return statement is supposed to convert the Python list object to a Numpy array object. However, the `numpy.array()` function has an optional parameter `copy`, which by default is set to `True`, so there is an extra copy.

Listing 3 from the NumpyCNN application [40], collects the weights of the layers in a convolutional neural network model into a Python list `network_weights` at line 2. The

code then converts the list to a Numpy array at line 4 using `numpy.array()`, which allocates a copy of the object. Since the `network_weights` list is never used after this function returns, there is no need to keep both the list and the Numpy array. For optimization, we use the `numpy.asarray()` API instead at line 4, which performs the in-place object conversion. This optimization reduces memory consumption by 42% and achieves a speedup of 3.6 $\times$.

C. Inefficiencies between CPU and GPU Layers

Many modern Python applications can interact directly with GPUs, such as creating data objects or offloading computations to GPUs. Generally, allocating and copying memory is necessary between CPU and GPU and does not follow COW. However, inefficiencies occur when Python creates duplicate objects on both CPU and GPU with unnecessary data copying. Listing 4, from the model factory in `torchbench` [41], uses the `torch.randn()` API to randomly generate `self.batch_size` tensors with dimension $3 \times 224 \times 224$ in CPU memory on line 3. These tensors are then copied to GPUs using the `to(self.device)` argument.

```

1 def __init__(self, model_name, test, device, ...):
2     ...
3     self.example_inputs = (torch
4         .randn((self.batch_size, 3, 224, 224))
5         .to(self.device), )
6+ self.example_inputs = (torch
7+     .randn((self.batch_size, 3, 224, 224),
8+         device=self.device)

```

Listing 4. Code snippet exemplifying cumbersome data movement from CPU to GPU. The `.to()` API copies data from CPU to GPU at line 5. However, the randomly generated tensor can be produced on GPU directly by setting the `device` argument as the code at line 8.

However, this CPU-GPU memory copy is unnecessary because one can directly allocate memory on GPU and assign the random values. Our optimization highlighted in line 4 explicitly sets the `device` argument, which avoids unnecessary CPU memory allocation and CPU-GPU data movement. This optimization reduces memory usage by 51% and results in an 8 $\times$ speedup for this function.

D. Common Features of Cross-Layer Inefficiencies

We identify several common features of various cross-layer memory inefficiencies that guide our design of DRPY.

- Cross-layer memory inefficiencies typically occur among individual memory objects. Therefore, rather than treating the entire memory as a single entity, an object-centric analysis is necessary to pinpoint the root causes of such inefficiencies accurately.
- Cross-layer memory inefficiencies manifest as redundant memory operations across the boundary of layers. Therefore, it is necessary to correlate memory behavior across layers to identify these inefficiencies.
- Detecting cross-layer memory inefficiencies requires fine-grained analysis of memory accesses and object values. Coarse-grained hotspot analysis of time and memory usage cannot pinpoint the root causes of such inefficiencies.

- Optimizing cross-layer memory inefficiencies requires simple API changes, such as adjusting API extensions or using different APIs. It is necessary for DRPY to pinpoint the problematic APIs for optimization.

IV. DRPY METHODOLOGY

The design principle of DRPY follows the common characteristics of various cross-layer memory inefficiencies. DRPY supports cross-layer, object-aware, and API-centric analysis, which includes program measurement, inefficiency analysis, and optimization guidance.

A. Measurement Basics: Cross-Layer Analysis

The foundation of DRPY lies in its ability to measure fine-grained memory accesses across different layers within a unified infrastructure. Achieving this requires DRPY to monitor both Python and native code execution. Given that both Python and native code are executed as binary streams via compilers or interpreters, DRPY directly monitors the binary execution, encompassing behaviors in both Python and native layers.

To implement this capability, we have built DRPY on top of DynamoRIO [42], a dynamic binary instrumentation engine capable of monitoring every binary instruction in detail, including operators, operands, and values in registers or memory. This solution ensures that DRPY can capture instruction-level behavior for fine-grained and cross-layer analysis.

However, DRPY faces two challenges posed by binary analysis: *Challenge I* involves distinguishing between program execution in Python layers and native layers, while *Challenge II* involves measuring Python-level behaviors rather than delving into the details of the Python interpreter. To address these challenges, DRPY intercepts the Python virtual machine to capture Python semantics.

1) *Intercepting the Python Virtual Machine:* Python, as an interpreted language, maintains detailed execution information within its virtual machine to address the aforementioned challenges. DRPY extracts this information by intercepting essential virtual machine operations and data structures through binary instrumentation, ensuring compatibility with standard Python and unmodified codebases. We use the popular CPython [43] as an example to illustrate the DRPY methodology, which can be applied to other Python implementations.

When Python calls a function, the CPython interpreter executes the internal function `_PyEval_EvalFrameDefault()` to establish a new frame and initialize a `_frame` data structure. DRPY intercepts this function, retrieving frame-related information. Additionally, DRPY creates a callback function passed to the frame's `f_trace` object, enabling precise measurement of Python bytecodes. It captures this callback for each bytecode execution and receives events defined in the Python virtual machine. DRPY performs two distinct actions in response to different events.

- DRPY captures events related to function calls and returns in Python code, allowing it to track control transfers between Python and native layers, addressing Challenge I.

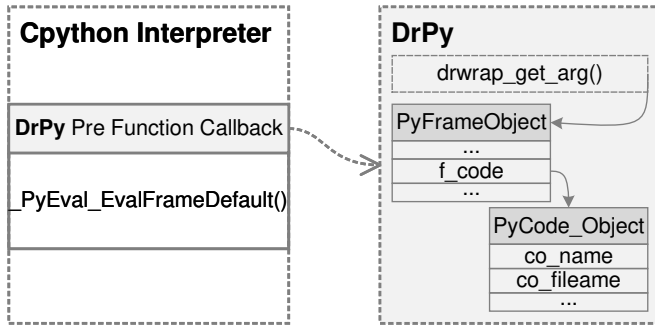


Fig. 2. Mechanism of the DRPY framework. When the CPython interpreter calls the `_PyEval_EvalFrameDefault()` function, the DRPY triggers `drwrap_get_arg()` function. We can get the `PyFrameObject` through the `drwrap_get_arg()` and then extract more detailed information at the bytecode level.

These events include `call` and `return`, which indicate Python layer function calls and returns, and `c_call`, `c_return`, and `c_exception`, which handle control transfers between Python and native layers.

- While in the Python layer, DRPY extracts opcode and operand information from these bytecodes and evaluates the values they use or produce. This approach tackles Challenge II, exclusively monitoring Python bytecode execution, filtering out binary instructions executed in the Python virtual machine.

2) *Monitoring Native Code*: When execution passes to the native layer, DRPY uses DynamoRIO's binary analysis to monitor each instruction. We can instrument any instruction to obtain operators, operands, and overload functions to capture arguments and return values. However, DRPY does not directly monitor GPU instruction execution. Instead, it monitors CPU-based GPU-related APIs, providing sufficient information for analysis, as described in the next section.

3) *Correlation between Different Layers*: In dynamic instrumentation, relating data objects across Python and native library layers with their memory addresses can be achieved using Python's `ctypes` module. The memory address of a Python object can be obtained with `ctypes.addressof()` by creating a C object that represents the Python object. To pass the memory address to the native library, define a C function that takes a pointer to the memory address as an argument and use `ctypes.byref()` to pass the address to the function. Ensure that the memory address remains valid and refers to a valid object.

B. Detection Algorithm: Object-Aware Analysis

DRPY applies an object-aware analysis to associate execution behaviors with specific objects. This is achieved by constructing an object flow chart that tracks the path of an object through different layers of execution, from the Python layer down to the native layer.

DRPY identifies Python objects by examining the local and global variable lists stored within Python frames. For objects allocated within the native layer, DRPY monitors functions

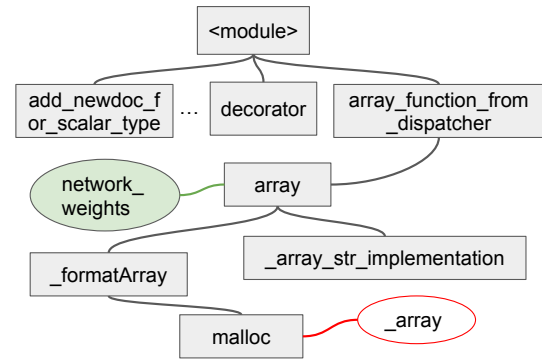


Fig. 3. Objects flow graph: Each ellipse represents a memory object, and each rectangle represents a function. Green nodes indicate that a memory object is created within the corresponding function. Red nodes indicate the presence of redundant copies of memory objects.

from the `malloc` family to identify them. It also overloads `cudaMalloc` to pinpoint objects allocated within GPU layers.

To correlate objects, DRPY checks for direct copies, categorizing them into value-agnostic patterns. It detects redundancies by verifying if a copy violates the Copy-On-Write (COW) principle.

Within the native layer, DRPY employs function overloading of `memcpy` and `cudaMemcpy` to determine relationships between source and destination objects. For correlating two Python objects, DRPY considers variables' names, types, and values across the program execution flow.

All monitored objects are organized in an object flow chart as shown in Figure 3. Each ellipse node represents a data object, and each rectangle node represents a function. Edges represent copy operations, with different edge colors indicating different data object allocation or copy operations.

Green edges indicate objects allocated in the called function, gray signifies objects referenced in subsequent functions, and red indicates objects that are hard-copied in the relevant function. The object flow chart forms the foundation of our inefficiency detection algorithm.

C. Detection Algorithm: Inefficiency Analysis

The object flow chart displays all object operations, and our goal is to identify unnecessary operations. Based on the inefficiency characterization in Section III, DRPY leverages two patterns for inefficiency detection:

- *Value-agnostic pattern*. This pattern applies when a copy operation occurs on the CPU, whether in Python or native layers. Such operations should adhere to the copy-on-write policy; otherwise, the object copy can be avoided by reusing the original objects or performing in-place data type conversion (e.g., from Python objects to NumPy objects). Detecting whether write operations occur on the object copy is necessary for identifying copy-on-write violations. Inefficiencies described in Sections III-A and III-B fall into this category.
- *Value-aware pattern*. This pattern applies to copy operations between CPU and GPU, which do not follow

the copy-on-write policy due to the requirements of the GPU programming model. However, a memory copy is unnecessary if the source object exhibits a clear value pattern, such as all the same values. In such cases, a memory copy can be avoided by directly manipulating the target object. The inefficiency described in Section III-C belongs to this category.

Algorithm 1 Redundant Copy Detection Algorithm

```

1: Initialization:
2: Let  $\mathcal{S}_{src}$  and  $\mathcal{S}_{dst}$  be stacks.
3: Let  $\mathcal{C}_{src}$ ,  $\mathcal{C}_{dst}$ , and  $\mathcal{R}$  be sets, initially empty.

4: function TRACEFUNC( $\mathcal{F}$ ,  $\mathcal{E}$ ,  $\mathcal{A}$ )
5:   if  $\mathcal{E} = \text{"call"}$  then
6:      $\mathcal{S}_{src}.push(\mathcal{C}_{src}); \mathcal{S}_{dst}.push(\mathcal{C}_{dst})$ 
7:      $\mathcal{C}_{src} \leftarrow \mathcal{F}_{globals} \cup \mathcal{F}_{locals}$ 
8:   end if
9:   if  $\mathcal{E} = \text{"opcode"}$  then
10:    if  $\mathcal{F}_{op} \in \text{COPY\_OPS}$  then
11:      Identify  $src \in \mathcal{F}_{bytecode}$  and  $dst \in \mathcal{F}_{bytecode}$ 
12:       $\mathcal{C}_{dst} \leftarrow \mathcal{C}_{dst} \cup \{dst\}$ 
13:      Update status of  $dst$  to "copied"
14:    else if  $\mathcal{F}_{op} \in \text{READ\_OPS} \cup \text{WRITE\_OPS}$  then
15:      Retrieve  $dst$  from  $\mathcal{F}_{bytecode}$ 
16:      if  $dst \in \mathcal{C}_{dst}$  then
17:        Update "write"  $\cup$  "read" status of  $dst$ 
18:      end if
19:    end if
20:  end if
21:  if  $\mathcal{E} = \text{"return"}$  then
22:    for all  $item \in \mathcal{C}_{dst}$  do
23:      if  $item$  is not written then
24:         $\mathcal{R} \leftarrow \mathcal{R} \cup \{F_{item}\}$ 
25:      end if
26:    end for
27:    Log the redundant set  $\mathcal{R}$  in file
28:    Restore previous  $\mathcal{C}_{dst}$  from  $\mathcal{S}_{dst}$ 
29:  end if
30: end function

```

1) *Analyzing value-agnostic patterns:* To detect redundancy in the value-agnostic pattern, we track copy operations in Python by examining execution types. Our method, as detailed in Algorithm 1, focuses on identifying object copy operations within Python's bytecode.

The detection process begins when the Python interpreter triggers the TRACEFUNC callback during function execution. This callback reveals the event type $\mathcal{E}$ at line 4 from its arguments. As the line 5 indicates, if the event is a `call`, it signifies the start of a function call. Here, DRPY saves the current lists of source objects ($\mathcal{C}_{src}$) and destination objects ($\mathcal{C}_{dst}$), and updates the source list ($\mathcal{C}_{src}$) accordingly.

As the program executes, Python decomposes instructions into bytecodes, each composed of operands and operators. An

opcode event is triggered in the callback each time a bytecode is processed. DRPY extracts the operand from the bytecode during this event. In Python, bytecode operands are categorized into sets, including those for read, write, and copy operations. At line 10, when an operand is part of the `COPY_OPS` set, DRPY identifies and records both the source and destination objects from the bytecode. The method is also used to track `READ_OPS` and `WRITE_OPS` at line 14. If an operand is identified as part of the copied object list ($\mathcal{C}_{dst}$), DRPY updates the operation status for that object at line 17.

Upon a `return` event, signaling the end of the function, DRPY examines each object in $\mathcal{C}_{dst}$ at line 22. It checks for the presence of write operations following the copy. If no write operation is detected on a copied object at line 23, it implies the copy was redundant. In such case, DRPY concludes that accessing the source object through reference, instead of copying it, would have been sufficient. Relevant information of the redundant object is stored in the redundant object set $\mathcal{R}$ at line 24.

2) *Analyzing value-aware patterns:* DRPY overloads `cudaMemcpy` to identify object copies across CPU and GPU. Although most such copies are necessary, in many cases, such copies can be avoided to save CPU-GPU communication. DRPY adopts value-based pattern recognition to detect redundant copies between CPU and GPU. DRPY dereferences the values of the source array of `cudaMemcpy` and identifies three value patterns for optimization: (1) an array with all the same values, (2) an array with all randomized values, and (3) an array with regular values according to indices. Any arrays with these three patterns can be directly allocated and initialized on GPUs, saving the CPU-GPU communication.

The `cudaMemcpy` function copies a specified number of bytes from the source memory address. To dereference these copied bytes, we need to determine the value type. DRPY extracts information such as source and destination memory addresses, byte size for copying, and CUDA memory copy types from the `cudaMemcpy` function call. When DRPY captures this function call, it first checks the copy types to ensure it's from CPU to GPU. Next, DRPY identifies the source object based on the source memory address and retrieves its data type. With the memory address, byte count, and data type, we can obtain the value for value-aware analysis.

DRPY recognizes three value patterns.

- *Constant pattern.* In this pattern, all items in the copied object have the same value. DRPY iterates all items in a copied object, checking if they are identical. If they are, the values of this object can be directly generated on target devices instead of being copied.
- *Diagonal pattern.* This pattern applies when elements in the copied object are separated by the same number of zeros, similar to a diagonal matrix. When the diagonal matrix is flattened in row major or column major, the elements with values are interspersed with a consistent number of zeros. DRPY counts the number of intermediate zeros to identify this pattern.

- *Random pattern.* In this pattern, the values in the copied object are irregular and random. Instead of checking values directly, DRPY examines the function names on relevant calling paths to determine if the source object was generated randomly. If the pattern does not fit into the constant or diagonal pattern categories, DRPY checks if the source object was created using randomizing APIs. If so, the copied object can be directly generated on the destination device.

D. Optimization Guidance: API-Centric Analysis

To generate actionable optimization guidance, DRPY attributes redundant copy operations to their source codes. These copy operations can be deeply nested within the call path across different layers. Obtaining the complete call path surrounding redundant copy operations is essential for identifying the problematic APIs that need optimization. For instance, in Listing 3, a `malloc` and a `memcpy` are invoked due to a redundant copy, but the optimization target is the NumPy API `numpy.array`. This optimization need arises because of a parameter `copy` with a default value of `True` in the `numpy.array` API, which triggers the invocation of the `memcpy` function in a lower hierarchy. Therefore, it is necessary to associate the call path with each redundant copy identified in the object flow graph, as shown in Figure 3.

Obtaining the full call path across Python and native layers is challenging. The Python interpreter employs the Global Interpreter Lock (GIL) implementation. The GIL is a mutual exclusion lock that allows only one thread to execute Python bytecodes at a time. In contrast, the native layer without GIL can have multiple threads executing simultaneously. Therefore, DRPY adopts distinct implementations to collect call paths in these two layers.

In the Python layer, DRPY constructs the call paths by traversing the Python frame stacks. The Python interpreter maintains a frame stack containing ample information. Each time a program executes a function, the Python interpreter switches to a new frame and pushes the previous function frame onto the stack. DRPY instruments a callback interface to capture function call and return events. When the callback reports a call event, DRPY adds a new node to the existing call path in the Python layer. When the callback announces a return event, DRPY moves back to the previous node on the call path.

In the native layer, DRPY creates local call paths for each thread. As we mentioned in section IV-A, when a program crosses from the Python layer to the native layer, the callback in the Python interpreter sets the event to `c_call`. DRPY creates a dummy leaf node on the call paths of the Python layer when the program enters the native layer. Once all local threads finish, DRPY attaches the root nodes of each thread's local call paths to the leaf node. This way, DRPY constructs the complete call paths in both the Python layer and the native layer.

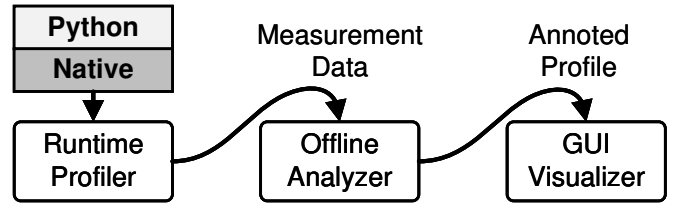


Fig. 4. Major Components of DRPY.

V. DRPY IMPLEMENTATION

Figure 4 outlines the workflow of DRPY, which includes an online profiler, an offline analyzer, and a GUI visualizer. The profiler interacts with both Python interpreter and native libraries to collect performance data, including tracking object copy operations, accesses, and full call paths. The offline analyzer aggregates profiles from different threads and generates a compact profile sorted by optimization potentials. The GUI visualizer annotates the object flow graph to present the analysis data to end-users for actionable optimizations. We elaborate on the implementation challenges in the following sections.

A. Runtime Profiler

DRPY leverages DynamoRIO to instrument Python applications. We implement DRPY on top of CPython 3.10 [43], taking advantage of CPython's open-source nature. We wrap `_PyEval_EvalFrameDefault()` to intercept Python operations and gather operator and operand information for analysis. In the Python layer, DRPY acquires the GIL to access the function frame information on the call stack.

When dealing with multithreading in the native layer, DRPY collects a profile for each thread independently to minimize synchronization overhead. We create thread-specific variables and data structures and construct call paths per thread. For the memory object accessed by all threads, DRPY records the operation in thread-local data structures. We use locks to aggregate operations on shared memory objects. If any thread performs a write operation on a shared memory object, DRPY acquires a lock on the object's status variable, updates it to "write", and maintains this status until the memory object's life cycle ends. Otherwise, the status remains "read", indicating that the memory object has been altered since its creation. As mentioned in Algorithm 1, DRPY would check if a copied memory object is modified at the exit of a function. This lock mechanism ensures the accuracy of determining whether a copied memory object is redundant.

B. Offline Analyzer and GUI

The offline analyzer selectively analyzes potential cross-layer memory inefficiencies on a subset of the monitored data objects. It is not worth the cost of checking a data object with only a single element to see if there exists another duplicate copy of it. Therefore, the offline analyzer primarily focuses on container data objects, such as `list`, `tuple`, `set`, and the other Python built-in data objects, as optimizing these more substantial data objects with relatively large memory blocks

can have a more significant impact on memory inefficiency elimination and application performance.

The offline analyzer produces an optimization that includes call paths from source data objects to destination data objects. Memory objects with call path information are stored in a hash table, using the memory address of raw data as the hash key to distinguish objects, and recording call paths as the hash value. When a new data object is created on an existing one without copying the raw data, they share the same raw data memory address. This means they have the same keys in the hash table, and DRPY will directly copy the hash value from the existing object to the new object instead of rebuilding the call paths from scratch.

VI. EVALUATION

We evaluate DRPY on several popular Python applications and frameworks to demonstrate DRPY's performance and capability. All experiments are done on a workstation with an Intel(R) Xeon(R) W-2295 CPU @3.00 GHz (16 cores, 32 threads, 128 GB memory) and NVIDIA GeForce RTX 3090 (2100 MHz, 24 GB GPU memory). We leverage DRPY to investigate multiple Python applications from various domains, including numerical computation libraries (Ta [44], NumpyCNN [40]), data analysis tools (Census_main [45], IrisData [46]), machine learning algorithms (CNN-from-scratch [47], SuperSloMo [48], Vision Framework [49], Autograd [50], LinearyRegression [51]), and deep learning frameworks (PyTorch [1]). The inputs of these applications are the default ones released together with them. We use CPython to run all these applications.

A. Optimization Results

DRPY is able to identify redundant memory operations in many Python applications as shown in Table I. This section summarizes the optimization results, and Section VII elaborates on how DRPY's output guides the optimization.

We present the function-level speedups and memory reductions to illustrate the performance improvements achieved. Function speedup rates vary across applications, with Ta and SuperSloMo achieving the highest speedup at $25.2\times$ and $23.1\times$ respectively, while Autograd and Fambench_xlrm report the lowest at $4.3\times$ and $3.9\times$, respectively. Memory speedups also vary but tend to be lower than function speedups. The highest memory speedup is reported by SuperSloMo at $1.98\times$, and the lowest by Census_main at $1.47\times$.

It is evident that optimization efforts have led to improved function execution speedups and memory usage across various applications, although the degree of improvement varies depending on the application and the specific API used.

B. Overhead

Table II presents the time overhead of DRPY, which averages at $21\times$. This level of overhead is expected due to the comprehensive instrumentation performed by the tool. DRPY's overhead can be broken down into two main components: (1) Instrumentation related to memory accesses and operations,

which is essential for detecting memory inefficiencies, and (2) instrumentation to calls and returns to obtain call paths for better optimization guidance. It is worth noting that the overhead of DRPY varies for different applications because different applications have different instruction mixes, resulting in different amounts of instrumented instructions for execution.

C. Discussions

Thanks to the exhaustive instrumentation, DRPY does not introduce any false positives or negatives. However, it is worth noting that not all inefficiencies identified by DRPY are optimizable. For example, some redundant memory operations are related to program inputs, so it is unsafe to remove them. Moreover, some memory copies are intentionally introduced to enjoy fast processing (e.g., copying data from CPU to GPU) or avoid security concerns. Thus, DRPY raises red flags about inefficiencies in programs, and users need to guarantee the correctness of the optimization.

VII. CASE STUDIES

We tested the DRPY on some popular modules and frameworks, including PyTorch, Numpy, Pandas, and scikit-learn, etc. The DRPY discovered some potential opportunities to optimize the code. The following subsections demonstrate the optimization by several cases.

A. Numpy Array

In the process of testing the Jax, we discover an inefficiency within the `parse_labels` function, specifically in the handling of array objects. The issue lies in the redundant copying from the built-in Python array object to the NumPy array object. Figure 5 shows the part of the object flow graph related to this issue. The graph with the red arrow highlights the redundant memory operations in the `_extendline` function. The ancestor of the `_extendline` function in the flow graph indicates the root cause —the improper use of `API array.array`.

The original version of the function uses the `array.array` built-in Python function, followed by a conversion to a NumPy array. This conversion entails an unnecessary memory copy operation, which can hinder performance, particularly when handling large arrays.

To optimize this operation and improve efficiency, we can modify the implementation by substituting the `array.array` and `np.array` operations with the `numpy.fromfile` function. The `numpy.fromfile` function reads data directly from

```
1 def parse_labels(filename):
2     with gzip.open(filename, "rb") as fh:
3         _ = struct.unpack(">II", fh.read(8))
4         return np.array(array.array("B", fh.read()),
5                           dtype=np.uint8)
6+      return np.fromfile(fh.read(), dtype=np.uint8)
```

Listing 5. Redundant copy from the built-in array class instance to Numpy array instance. The original code at line 4 packs the file stream into an instance of `array.array`, and then converts it into a `numpy.array` instance. The conversion incurs an unnecessary copy of the array object.

TABLE I
EVALUATION OF FUNCTION EXECUTION TIME, MEMORY USAGE, AND CORRESPONDING SPEEDUPS FOR MODELS IN PYTORCH BENCHMARK, JAX, SCIKIT-LEARN, ETC. INDUSTRIAL PYTHON APPLICATIONS.

Applications	Problem Code	Related Frameworks	APIs	Function Speedup	Memory Speedup	Inefficiency Pattern	Optimization
Ta	model.py main.py	Numpy	np.meshgrid()	25.2×	1.66×	value agnostic	generate the meshgrid on target device directly
NumpyCNN	numpycnn.py	Numpy	np.array() np.uniform()	7.2×	1.52×	value agnostic	remove the np.array() wrapper
Census_main	ftrl.py	Numpy	np.array() np.asarray()	12.4×	1.47×	value agnostic	use np.asarray() directly
IrisData	nn_backprop.py	Numpy	np.array() np.view()	7.6×	1.88×	value agnostic	use np.view() directly
CNN-from-scratch	conv.py	Pandas	pd.index	10.5×	1.63×	value agnostic	set the copy_on_write mode
Metaheuristics	FunctionalUtil.py Epo.py	Pandas	pd.iloc()	9.3×	1.77×	value agnostic	set the copy_on_write mode
Fourier-Transform	transforms.py	Pandas	pd[]	9.2×	1.84×	value agnostic	set the copy_on_write mode
Super SloMo	model.py main.py	PyTorch	np.meshgrid() torch.meshgrid()	23.1×	1.98×	value agnostic	generate the meshgrid on target device directly
Vision Framework	model_factory.py	PyTorch	np.meshgrid() torch.meshgrid()	4.8×	1.82×	value agnostic	generate the meshgrid on target device directly
Autograd	main.py	PyTorch	torch.empty_like()	4.3×	1.76×	value aware	generate the tensor on target device directly
Fambench_xlmr	functional.py	PyTorch	torch.zeros_like()	3.9×	1.58×	value aware	generate the zero tensor on target device directly
Jax Test	test.py bench.py	Jax	Array.array() np.array()	21.3×	1.73×	value agnostic	remove the array wrapper
LinearRegression	LinearRegression.py	scikit-learn	model.fit()	11.5×	1.82×	value aware	set the copy parameter to false

TABLE II
TIME OVERHEAD OF THE DRPY.

Application	Original Time	Profiling Time	Overhead
Ta	13.43 ms	282.03 ms	21×
NumpyCNN	15.66 ms	454.14 ms	29×
Census_main	8.81 ms	193.82 ms	22×
IrisData	723.92 ms	1.52 s	21×
CNN-from-scratch	141.06 ms	1.83 s	13×
Metaheuristics	50.76 ms	406.08 ms	28×
Fourier-Transform	16.66 ms	466.48 ms	28×
Super SloMo	346.11 ms	5.19 s	15×
Vision Framework	59.28 ms	1.78 s	30×
Autograd	41.50 ms	1.16 s	28×
Fambench_xlmr	21.84 ms	327.6 ms	15×
JAX Test	257.43 ms	4.63 s	18×
Linear Regression	2.80 ms	44.8 ms	16×
Geometric mean	—	—	21×

the file into a NumPy array, thus eliminating the redundant array object copy. This approach reduces the memory usage by 49% incurred by the built-in `array.array` API.

B. FAMBench Model

DRPY detects redundant tensor object copy from CPU to GPU in the `dropout` function of the FAMBench model.

The `fambench_xlmr` model in the PytorchBenchmark set calls the `dropout` function tremendous times to randomly change the numerical value in tensors to 0. The `dropout` function adopts the Bernoulli distribution to sample the values of the tensor. The change does not happen at the Python level, instead, it needs to pass the tensor into the lower CUDA

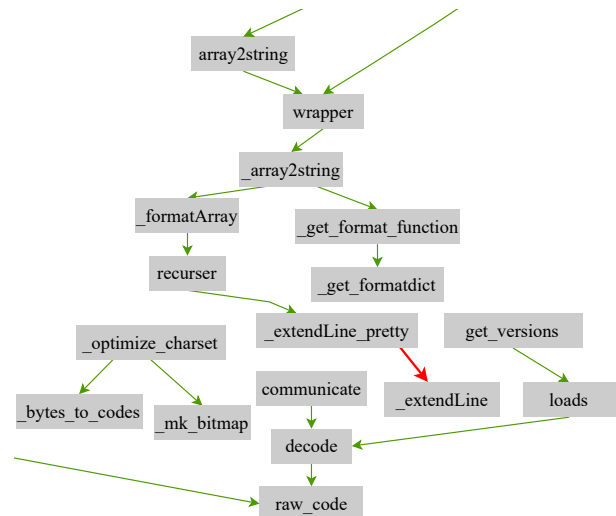


Fig. 5. Part of objects flow graph of numpy array case relating to Listing 5. The red arrow indicates there exists a redundant copy generated in the `_extendLine` function call.

library level and then invoke the corresponding function, for example, the `aten::zeros()` to modify the tensor. When we run our DrPy tool to profile the `fambench_xlmr` model, the diagnostic log indicates there is $\frac{1}{3}$ of `dropout` function calls either change the tensor to all zeros or stay the same. The phenomenon suggests $\frac{1}{3}$ of the possibility values, p , generated by the Bernoulli distribution are either 0 or 1. When the possibility value p becomes 1, the `dropout` function changes

all the values in the tensor to 0. Otherwise, when the possibility value p becomes 0, the dropout function does not modify any piece of the tensor. There is no need to invoke the low-level CUDA function calls under these two special occasions. The PyTorch library has already provided the API (application programming interface), `zeros_like()`, to zero tensors. The `zeros_like()` can achieve the same functionality without invoking the lower-level function call at the CUDA layer, and therefore it can help avoid the interaction inefficiency.

The dropout function in listing 6 randomly zeros some of the elements of the input tensor with probability p using samples from a Bernoulli distribution. If the value of p is 1, it means the Bernoulli distribution samples all the elements in the tensor and then change them to zeros. If the value of p is 0, it means the Bernoulli distribution samples nothing from the tensor and therefore does not modify a penny of the tensor. According to the semantics of the dropout function, we add the branch statement, `if` and `elif` from line 6 to line 9 in the listing 6 to handle the two special cases.

```
1 def dropout(input: Tensor, p: float = 0.5, training:
    bool = True, inplace: bool = False) -> Tensor:
2     if has_torch_function_unary(input):
3         return handle_torch_function(dropout, (input,),
            input, p=p, training=training, inplace=
            inplace)
4     if p < 0.0 or p > 1.0:
5         raise ValueError("dropout probability has to be
            between 0 and 1, " "but got {}".format(p))
6+    if p == 0.0:
7+        return input
8+    elif p == 1.0:
9+        return torch.zeros_like(input)
10    return _VF.dropout_(input, p, training) if
11    inplace else _VF.dropout(input, p, training)
```

Listing 6. The dropout function makes a special case check to save unnecessary interaction inefficiency between high level function call at the Python layer and the lower function call the CUDA layer.

The dropout function in listing 6 randomly zeros some of the elements of the input tensor with probability p using samples from a Bernoulli distribution. If the value of p is 1, it means the Bernoulli distribution samples all the elements in the tensor and then change them to zeros. If the value of p is 0, it means the Bernoulli distribution samples nothing from the tensor and therefore does not modify a penny of the tensor. According to the semantics of the dropout function, we add the branch statement, `if` and `elif` from line 6 to line 9 in the listing 6 to handle the two special cases.

The `fambench_xlmr` model uses less 5% time with the optimized dropout function. The dropout function is not bundled in the `fambench_xlmr` model, it is a library function implemented in the `torch.nn` module. The modification on the dropout function not only benefits the `fambench_xlmr` model, but also bring advantage to all the models relying on the dropout function.

C. Pandas Dataframe

In pandas, a DataFrame copy is typically created only when necessary to prevent excessive memory usage. This behavior

is implemented using the aforementioned technique, “Copy-on-Write” (COW).

However, specific operations in Pandas contravene the COW principle, leading to data duplication even without any modifications to the original dataset. This unexpected duplication can result in increased memory usage and reduced performance.

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame(np.random.randn(5, 3), columns=[
    'a', 'b', 'c'])
5 mask = df['a'] > 0
6- subset = df[mask]
7+ subset = df.loc[mask, :]
```

Listing 7. Code snippet demonstrating unnecessary copy of dataframe. These two code versions generate exactly the same subset. However, the upper version (Line 6) consumes more than 2 times of memory than the lower version (Line 7), and the upper one needs more time than the lower one.

One such operation is indexing with a boolean array. As shown in Listing 7, when using the boolean array (`mask`) to filter rows with values exceeding 0, Pandas creates a copy of the data, even if it is unmodified. This is because the resulting subset requires a unique index, separate from the original DataFrame. While this maintains index accuracy, it increases memory usage and can slow down performance, especially with large DataFrames or frequent similar operations. The code is optimized by using `DataFrame.loc` for selection by boolean array. This method reduces the memory usage by up to 62%.

VIII. CONCLUSIONS

This paper presents DRPY, a novel profiler designed to uncover memory inefficiencies across various layers in modern Python applications. In contrast to existing Python profilers, DRPY employs exhaustive instrumentation to meticulously monitor Python operations and binary instructions. Leveraging the wealth of runtime information, DRPY is capable of identifying a range of previously unrecognized memory inefficiencies, particularly redundant memory copy operations. Furthermore, DRPY provides object flow graphs, call paths, and source code attribution, facilitating intuitive optimization guidance. Through an evaluating of DRPY on 13 modern Python applications, we successfully identified and optimized numerous inefficiencies, guided by the insights provided by DRPY. This optimization effort resulted in significant performance improvements.

ACKNOWLEDGEMENTS

We thank anonymous reviewers and our shepherd for their valuable comments. This research is partially supported by NSF CNS 2050007 and a Google gift.

ARTIFACT APPENDIX

A. Abstract

This artifact contains the implementation of a profiler, to pinpoint cross-layer inefficiencies. Moreover, this artifact contains the scripts and reference implementations required

to perform the evaluation experiment presented in the paper, DRPY: *Pinpointing Inefficient Memory Usage in Multi-Layer Python Applications*.

B. Artifact check-list (meta-information)

- **Algorithm:** Redundant copy detection.
- **Program:** The DRPY implemented in C++ and Python. Evaluation programs are included.
- **Compilation:** We provide build scripts via `gcc/g++` and `Python`.
- **Transformations:** Dynamic binary instrumentation. It is public and included.
- **Binary:** Included for Linux. Source code and scripts included to regenerate binaries.
- **Run-time environment:** Linux with Python (≥ 3.8), `gcc` (≥ 4.8), `CMake` (≥ 3.12), `binutils` (≥ 2.26), `CUDA` (≥ 11.8), `Perl`.
- **Hardware:** GPU needed to run some evaluation applications.
- **Output:** Flow graphs to show detection results. Numerical speedup and memory usage.
- **Experiments:** docker clone and run; run build scripts; run evaluation scripts; observe performance results
- **Publicly available?:** Yes

C. Description

1) *How to access:* Our artifact is hosted on Docker Hub at: <https://hub.docker.com/r/captaincui/drpy>

2) *Hardware dependencies:* A CUDA-enabled GPU is recommended to run several evaluation applications. In the paper, we conduct experiments on a workstation with NVIDIA GeForce RTX 3090, an Intel(R) Xeon(R) W-2295 CPU @3.00 GHz (16 cores, 32 threads), and 128GB memory.

3) *Software dependencies:* DRPY requires Python 3, DrCCTProf, and a functional C++ compiler as its main dependencies. Complete software dependencies are described on the DockerHub page. A docker image is provided encapsulating all enlisted dependencies.

D. Installation

DRPY can be installed by executing the build script in the docker image and then setting the PATH environment variable.

```
1$ docker pull captaincui/drpy
2$ docker run -t -d --name drpy-dev captaincui/drpy
3$ docker exec -it --user ubuntu drpy-dev zsh
4$ cd ~/workspace/drcctprof_clients
5$ ./build.sh
6$ export drrun=/home/ubuntu/workspace/
   drcctprof_clients/DrCCTProf/build/bin64/drrun
```

Our artifact is one of the clients of the DrCCTProf framework. After executing the installation commands, DRPY is capable being executed from the console directly. Detailed installation and usage descriptions are given on the Docker Hub page.

E. Experiment workflow

After building and installing DRPY and relevant Python packages, provided scripts should be used for running evaluation applications.

Detailed descriptions for the experiment workflow are provided on the Docker Hub page.

F. Evaluation and expected results

The main result of the artifact evaluation process is the generation of value flow graphs, as illustrated in Figure 5 of the paper, for the tested applications. These graphs visually represent the detected redundancies, simplifying the optimization process for developers.

The evaluation package includes both the original and optimized versions of the applications. Reviewers are encouraged to benchmark these two versions to assess the speedup and memory reduction achieved. Depending on the similarity of the hardware used for evaluation, speedups of comparable magnitude and trends should be observed.

REFERENCES

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [2] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. (2015) TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [3] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. (2018) JAX: composable transformations of Python+NumPy programs. [Online]. Available: <http://github.com/google/jax>
- [4] H. Jiang, D. Sun, V. Jampani, M.-H. Yang, E. Learned-Miller, and J. Kautz, "Super slo-mo: High quality estimation of multiple intermediate frames for video interpolation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 9000–9008.
- [5] J. Roskind and G. V. Rossum. (1994) cProfile - Deterministic profiling tools of Python programs. Software available from Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/profile.html>
- [6] Y. Zhu. (2019) Guppy3: A Python Programming Environment & Heap analysis toolset. [Online]. Available: <https://zhuyifei1999.github.io/guppy3/>
- [7] B. Frederickson, Y. Goldschmidt, A. Khramov, M. Lv, C. Heer, A. Johnson, A. Shorin, M. Streeter, E. Imeys, N. Rubinstein, C. Clauss, C. Wong, G. Joseph, S. Walker, A. Meyre, M. F. Schönlitz, D. Vo, Y. Xie, A. Lee, T. Gates, J. Gmach, D. Martynov, T. Bechtold, C. Craft, G. Ayoub, F. Aalund, T. Waldmann, and A. Chanda. (2018) py-spy: Sampling profiler for Python programs. [Online]. Available: <https://github.com/benfred/py-spy>
- [8] E. Klitzke, I. Nikolaev, J. Pickering, T. Joynson, J. Devanur, T. Konick, A. Jeffery, S. Karas, J. Vreeken, M. Bolingbroke, A. Beutot, Z. Li, L. Lausen, J. Phillips, faicker, F. Micea, S. Zheng, Oleg, and E. Freeman. (2016) Pyflame: A PTracing Profiler For Python. [Online]. Available: <https://github.com/uber-archive/pyflame>

- [9] J. Rickerby, G. Oxberry, M. Klein, I. Aaronsohn, A. Meurer, Tian, A. Shaw, J. Squire, C. Clauss, C. Hudon, Steven, R. van Hattem, A. G. Badaracco, B. Rostás, G. Kostyuk, M. Udoff, M. Graham, T. Aarsen, W. Li, M. OBrien, A. Bazzanella, B. Beasley, C. Anand, D. Vanderkam, J. Danjou, K. K. Viswanathan, M. Fiedler, Pallak, and Pavka. (2014) pyinstrument: Call stack profiler for Python. Shows you why your code is slow! [Online]. Available: <https://github.com/joerick/pyinstrument>
- [10] Gak, M.-A. Bisotti, F. C. B. da Luz, A. Todorov, B. V., D. J. Felix, D. Turnbull, R. Bednark, V. Markovtsev, and R. Bubley. (2009) Python Call Graph is a Python module that creates call graph visualizations for Python applications. [Online]. Available: <https://github.com/gak/pycallgraph>
- [11] V. Pelletier, P. Cuvilliers, P. A., and X. Zeng. (2013) Line-granularity, thread-aware deterministic and statistic pure-python profiler. [Online]. Available: <https://github.com/vpelletier/pprofile>
- [12] F. Pedregosa, P. Gervais, K. Altendorf, D. Ryzhykau, A. Foucault, D. Novozhilov, J. L. C. Rodríguez, C. Clauss, B. Bengfort, M. Becker, Carlo, V. Niculae, sagar UDAY KUMAR, Toni, O. Grisel, D. Hnyk, F. Xicluna, M. H. Tariq, I. Ozsvald, W. Silversmith, D. McDonald, I. Mikhelson, J. Guyer, M. H. Gerlach, M. Korobov, P. Bowyer, M. Pieters, A. Menon, justhalf, and E. Larson. (2011) Monitor Memory usage of Python code. [Online]. Available: https://github.com/pythonprofilers/memory_profiler
- [13] G. N. Tornetta, A. Shaw, and S. Lishak. (2020) Austin: A Python Frame Stack Sampler for CPython. [Online]. Available: <https://github.com/P403n1x87/austin-vscode>
- [14] J. Reinders, *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*, ser. Engineer to Engineer Series. Intel Press, 2005. [Online]. Available: <https://books.google.com/books?id=ArwZPQAACAAJ>
- [15] A. C. De Melo, "The new linux perf tools," in *Slides from Linux Kongress*, vol. 18, 2010.
- [16] A. Nistor, L. Song, D. Marinov, and S. Lu, "Toddler: Detecting performance problems via similar memory-access patterns," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 562–571.
- [17] L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G. Marin, J. Mellor-Crummey, and N. R. Tallent, "HPCToolkit: Tools for performance analysis of optimized parallel programs," *Concurrency and Computation: Practice and Experience*, vol. 22, no. 6, pp. 685–701, 2010.
- [18] S. Wen, X. Liu, J. Byrne, and M. Chabbi, "Watching for software inefficiencies with witch," in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, 2018, pp. 332–347.
- [19] S. Wen, M. Chabbi, and X. Liu, "Redspy: Exploring value locality in software," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 47–61.
- [20] M. Chabbi and J. Mellor-Crummey, "DeadSpy: A Tool to Pinpoint Program Inefficiencies," in *Proceedings of the Tenth International Symposium on Code Generation and Optimization*, ser. CGO '12. New York, NY, USA: ACM, 2012, pp. 124–134.
- [21] P. Su, S. Wen, H. Yang, M. Chabbi, and X. Liu, "Redundant Loads: A Software Inefficiency Indicator," in *Proceedings of the 41st International Conference on Software Engineering*, ser. ICSE '19. IEEE Press, 2019, p. 982–993. [Online]. Available: <https://doi.org/10.1109/ICSE.2019.00103>
- [22] E. D. Berger, S. Stern, and J. Altmayer Pizzorno. (2023, Jul.) Triangulating Python Performance Issues with Scalene.
- [23] J. Tan, Y. Chen, Z. Liu, B. Ren, S. L. Song, X. Shen, and X. Liu, "Toward Efficient Interactions between Python and Native Libraries," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 1117–1128. [Online]. Available: <https://doi.org/10.1145/3468264.3468541>
- [24] M. Davis, S. maintainers, and contributors. (2021, Nov.) SnakeViz, a browser based graphical viewer for Python profiling data. [Online]. Available: <https://jiffyclub.github.io/snakeviz>
- [25] T. Ball and J. Daniel, "Deconstructing Dynamic Symbolic Execution," in *Dependable Software Systems Engineering*, ser. NATO Science for Peace and Security Series, D: Information and Communication Security, M. Irlbeck, D. A. Peled, and A. Pretschner, Eds. IOS Press, 2015, vol. 40, pp. 26–41. [Online]. Available: <https://doi.org/10.3233/978-1-61499-495-4-26>
- [26] M. Bann. (2016) pySym: Python Symbolic Execution. <https://github.com/bannsec/pySym>. [Online]. Available: <https://github.com/bannsec/pySym>
- [27] I. S. Cordasco. (2010) Flake8: Your Tool For Style Guide Enforcement. <https://flake8.pycqa.org/en/latest/>.
- [28] T. Crosley. (2014) Frosted documentation. <https://pypi.org/project/frosted/>.
- [29] H. Gulabovska and Z. Porkoláb, "Survey on Static Analysis Tools of Python Programs," in *SQAMIA*, 2019.
- [30] X. Team *et al.* (2017) XLA: Optimizing Compiler for Machine Learning.
- [31] T. Chen, T. Moreau, Z. Jiang, H. Shen, E. Q. Yan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: end-to-end optimization stack for deep learning," *arXiv preprint arXiv:1802.04799*, vol. 11, p. 20, 2018.
- [32] W. Zhou, Y. Zhao, G. Zhang, and X. Shen, "HARP: Holistic Analysis for Refactoring Python-Based Analytics Programs," in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 506–517.
- [33] P. Team. (2022) PyTorch Profiler: Determine the Most Expensive Operators in the Model. <https://github.com/pytorch/pytorch/blob/main/torch/profiler/profiler.py>.
- [34] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. (2018) JAX Profiler: Profiling JAX Programs. . [Online]. Available: <http://github.com/google/jax>
- [35] Q. Xu, Yin, T. Li, J. Wang, J. Baiocchi, C. Verghese, J. Choi, J. Sun, S. Yi, R. Ślawik, E. Martin, Luke, N. Felt, R. Narasimhan, S. Cai, Vivian, W. Cromar, Y. D. Yang, and A. Fehlnér. (2020) A profiling and performance analysis tool for TensorFlow. [Online]. Available: <https://github.com/tensorflow/profiler>
- [36] L. Song and S. Lu, "Performance Diagnosis for Inefficient Loops," in *Proceedings of the 39th International Conference on Software Engineering*, ser. ICSE '17. IEEE Press, 2017, p. 370–380. [Online]. Available: <https://doi.org/10.1109/ICSE.2017.41>
- [37] J. Tan, S. Jiao, M. Chabbi, and X. Liu, "What Every Scientific Programmer Should Know about Compiler Optimizations?" in *Proceedings of the 34th ACM International Conference on Supercomputing*, ser. ICS '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3392717.3392754>
- [38] X. You, H. Yang, K. Lei, Z. Luan, and D. Qian, "VCLinic: A Portable and Efficient Framework for Fine-Grained Value Profilers," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 892–904. [Online]. Available: <https://doi.org/10.1145/3575693.3576934>
- [39] T. pandas development team. (2020, Feb.) pandas-dev/pandas: Pandas. [Online]. Available: <https://doi.org/10.5281/zenodo.3509134>
- [40] A. Gad. (2018) Numpycnn: Implementing convolutional neural networks from scratch. <https://github.com/ahmedfgad/NumPyCNN>.
- [41] Y. Hao, X. Zhao, B. Bao, D. Berard, W. Constable, A. Aziz, and X. Liu, "Torchbench: Benchmarking pytorch with high api surface coverage," *arXiv preprint arXiv:2304.14226*, 2023.
- [42] Q. Zhao, X. Liu, and M. Chabbi, "DRCCTPROF: A Fine-Grained Call Path Profiler for ARM-Based Clusters," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020, pp. 1–16.
- [43] Python Core Team (2015). (2015) Python: A dynamic, open source programming language. Python Software Foundation. [Online]. Available: <https://www.python.org/>
- [44] D. L. Padiál. (2018) Technical analysis library in python. <https://github.com/bukosabino/ta>.
- [45] Stasi. (2018) Numpywdl: Implement wide & deep algorithm by using numpy. <https://github.com/stasi009/NumpyWDL>.
- [46] L. Stott. (2017) Irisdata: Iris data example python numpy. <https://github.com/leestott/IrisData>.
- [47] V. Zhou. (2019) A convolution neural network (cnn) from scratch. <https://github.com/vzhou842/cnn-from-scratch>.
- [48] H. Jiang, D. Sun, V. Jampani, M. Yang, E. G. Learned-Miller, and J. Kautz, "Super sloMo: High quality estimation of multiple intermediate frames for video interpolation," *CoRR*, vol. abs/1712.00080, 2017. [Online]. Available: <http://arxiv.org/abs/1712.00080>

- [49] T. maintainers and contributors. (2016, Nov.) TorchVision: PyTorch's Computer Vision library. [Online]. Available: <https://github.com/pytorch/vision>
- [50] T. A. maintainers and contributors. (2017) Automatic differentiation package - torch.autograd. <https://pytorch.org/docs/stable/autograd.html>.
- [51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.